

LOG ROTATION

Concepts · Why We Use It · Practical Guide

What is Log Rotation?

Log rotation is the automated process of managing log files by periodically archiving, compressing, and deleting old log data to prevent files from growing indefinitely and consuming all available disk space.

Why We Use Log Rotation

Disk Space Management

Log files grow continuously. A busy web server can generate gigabytes daily. Without rotation, disks fill up causing crashes or system failures.

Performance

Reading/writing to a massive single log file is slow. Smaller, segmented files are faster to parse and search.

Easier Debugging

Organised logs (e.g. one file per day) make it easy to isolate issues to a specific time window.

Compliance & Retention

HIPAA, GDPR, PCI-DSS require logs to be retained for a specific period — not forever, and not deleted too soon.

Backup & Archiving

Smaller rotated files are easier to compress, archive, and back up.

Practical Guide — logrotate on Linux

```
# Check installation
logrotate --version

# View global config
cat /etc/logrotate.conf
ls -la /etc/logrotate.d/

# Check current log sizes (CRITICAL on EC2 – default root is only 8GB!)
du -sh /var/log/*
df -h
```

Sample logrotate Config

```
/var/log/myapp/*.log {
    daily                # Rotate every day
    missingok            # Don't error if log file is missing
    rotate 14            # Keep last 14 rotated files
```

```
    compress                # Compress old logs with gzip
    delaycompress           # Compress previous rotation, not current
    notifempty              # Don't rotate if log is empty
    create 0640 www-data adm
    sharedscripts
    postrotate
        systemctl reload myapp > /dev/null 2>&1 || true
    endscript
}
```

Test & Force Rotation

```
# Dry run – shows what WOULD happen
sudo logrotate --debug /etc/logrotate.d/myapp

# Force rotation NOW
sudo logrotate --force /etc/logrotate.d/myapp

# Verify results
ls -lh /var/log/myapp/
```

Common Rotation Strategies

Strategy	Config Keyword	Use Case
Daily	daily	High-traffic apps
Weekly	weekly	Low-traffic / batch apps
Monthly	monthly	Archival logs
By Size	size 100M	Unpredictable log volume
Size + Daily	daily + maxsize 50M	Best of both worlds

Framework Examples

Python — RotatingFileHandler

```
from logging.handlers import RotatingFileHandler

handler = RotatingFileHandler(
    'app.log',
    maxBytes=10 * 1024 * 1024, # 10 MB
    backupCount=5
)
```

Node.js — winston DailyRotateFile

```
const { createLogger, transports } = require('winston');
require('winston-daily-rotate-file');

const logger = createLogger({
  transports: [new transports.DailyRotateFile({
    filename: 'logs/app-%DATE%.log',
```

```
    datePattern: 'YYYY-MM-DD',
    maxSize: '20m',
    maxFiles: '14d'
  }]
});
```

Docker Compose

```
services:
  app:
    logging:
      driver: "json-file"
      options:
        max-size: "10m"
        max-file: "5"
```

Key Takeaways

- Always set a retention limit (rotate N) — never keep logs forever unless required.
- Always compress old logs — gzip achieves 90%+ compression on text logs.
- Use postrotate scripts to signal your app to reopen log file handles.
- Monitor disk usage even with rotation — alert at 70-80% disk usage.
- Always test with --debug before relying on a new config in production.